

Module 3: AI for Mechatronics

ANN, MLP, Backpropagation, CNN, RNN, Autoencoders and Training Strategies

Dr. Rajan Prasad

SMEC
VIT University

July 23, 2026

Modern mechatronic systems generate large volumes of complex data.

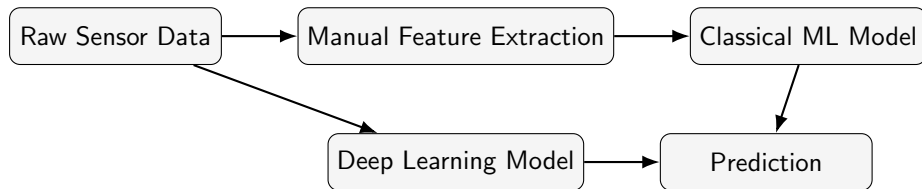
Data Sources

- Camera images
- Vibration signals
- Motor current and voltage
- IMU and encoder data
- Force/torque sensors
- Robot trajectories

Deep Learning Tasks

- Visual inspection
- Fault diagnosis
- Predictive maintenance
- Robot perception
- Sensor fusion
- Control policy learning

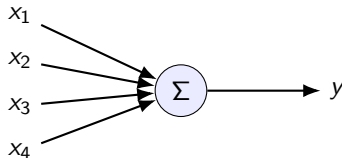
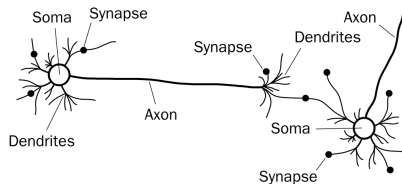
Key idea: Deep networks learn features directly from data.



- Classical ML often depends on hand-crafted features.
- Deep learning learns hierarchical representations automatically.
- This is useful for images, signals, speech, trajectories and large datasets.

Biological neuron

- Receives input from other neurons.
- Processes signals.
- Fires when activation is sufficiently strong.



Artificial neural networks are simplified mathematical models inspired by biological neurons.

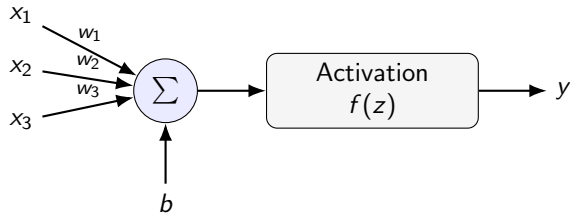
Reference: Negnevitsky, M., *Artificial Intelligence: A Guide to Intelligent Systems*, Pearson, 4th Edition, 2021.

An artificial neuron computes:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

$$y = f(z)$$

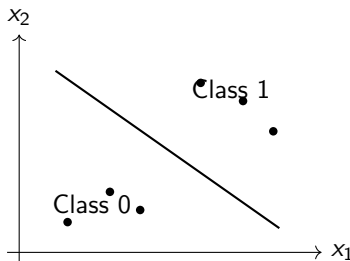
- x_i : inputs
- w_i : weights
- b : bias
- f : activation function
- y : output



Biological NN	Artificial NN
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

- A perceptron is a single artificial neuron used for binary classification.
- It computes a weighted sum and applies a threshold function.

$$y = \begin{cases} 1, & w^T x + b \geq 0 \\ 0, & w^T x + b < 0 \end{cases}$$



A single perceptron can only solve linearly separable problems.

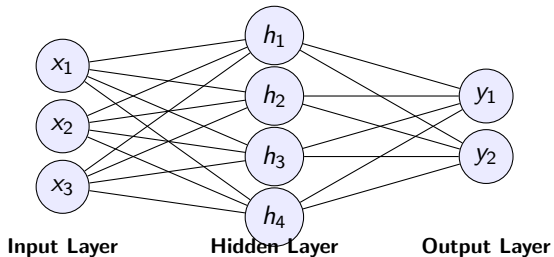
Can solve:

- AND gate
- OR gate
- Linear decision boundaries

Cannot solve directly:

- XOR gate
- Nonlinear classification
- Complex robot perception tasks

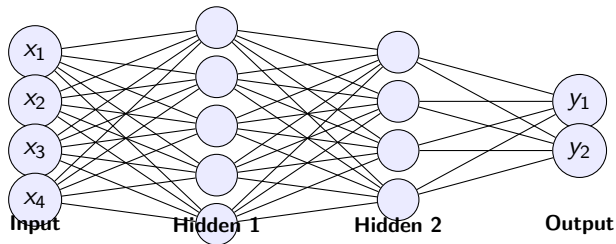
Solution: Use hidden layers and nonlinear activation functions.



- Input layer receives sensor/features.
- Hidden layers learn internal representations.
- Output layer produces prediction, class or control decision.

Application Area	ANN Use
Motor fault diagnosis	Healthy/Faulty classification
Robot perception	Object recognition
Manipulator control	Inverse kinematics approximation
Autonomous vehicles	Lane and object detection
Manufacturing	Surface defect detection
Smart sensors	Signal classification
Prosthetics	EMG-based movement prediction

An MLP is a feedforward neural network with one or more hidden layers.

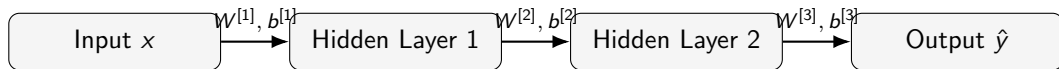


MLPs are suitable for tabular data, regression and classification.

For layer l :

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = f(z^{[l]})$$



Forward propagation computes predictions from input to output.

Example: Predict motor temperature from sensor data.

Inputs:

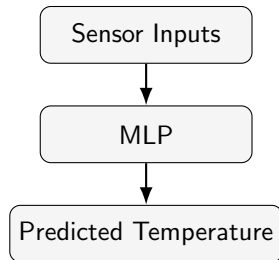
- Current
- Voltage
- Speed
- Ambient temperature
- Load torque

Output:

$$\hat{T}_{motor}$$

Loss function:

$$MSE = \frac{1}{N} \sum_{i=1}^N (T_i - \hat{T}_i)^2$$



Example: Classify machine health from vibration features.

Input Features	Output Class
RMS vibration, FFT peaks, current harmonics	Healthy
RMS vibration, FFT peaks, current harmonics	Bearing fault
RMS vibration, FFT peaks, current harmonics	Rotor imbalance

For multiclass classification, the output layer often uses Softmax:

$$P(y = k|x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

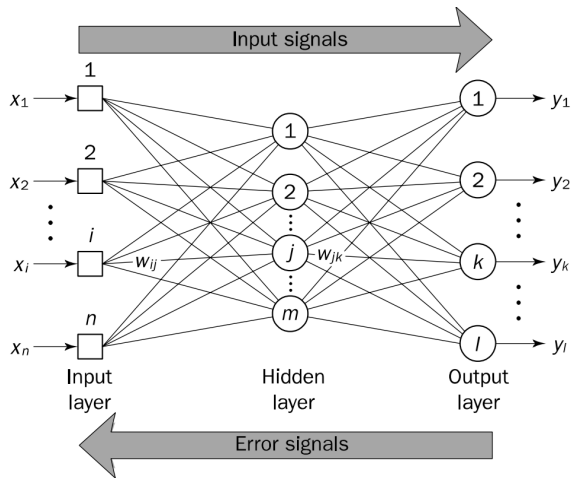
Design choices

- Number of hidden layers
- Number of neurons per layer
- Activation functions
- Loss function
- Optimizer
- Learning rate

Important: More layers do not always mean better generalization.

Engineering guideline

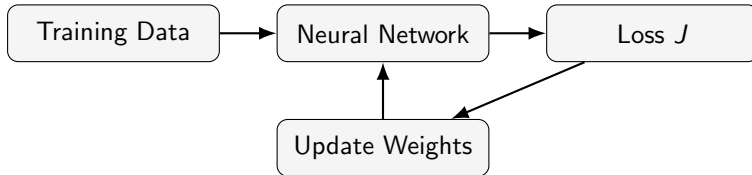
- Start simple.
- Increase complexity only if validation error is high.
- Use regularization when overfitting occurs.
- Use validation data for model selection.



Reference: Negnevitsky, M., *Artificial Intelligence: A Guide to Intelligent Systems*, Pearson, 4th Edition, 2021.

Neural networks contain many weights and biases. Training means finding values that minimize prediction error.

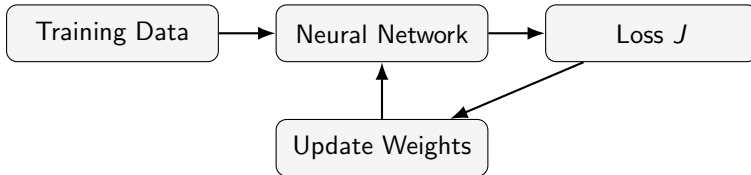
$$\theta^* = \arg \min_{\theta} J(\theta) =$$



Backpropagation computes gradients efficiently using the chain rule.

Neural networks contain many weights and biases. Training means finding values that minimize prediction error.

$$\theta^* = \arg \min_{\theta} J(\theta) =$$



Backpropagation computes gradients efficiently using the chain rule.

Think:)

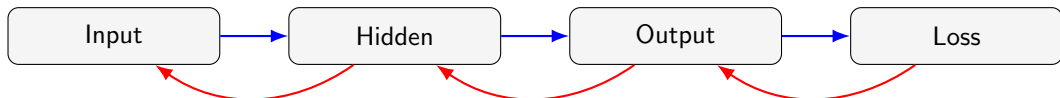
Does Backpropagation happen in biological world?

Forward Pass

- Input data enters the network.
- Each layer computes activations.
- Network produces prediction.
- Loss is calculated.

Backward Pass

- Error is propagated backward.
- Gradients are computed using chain rule.
- Weights and biases are updated.



$$w_{jk}(p+1) = w_{jk}(p) - \Delta w_{jk}(p)$$

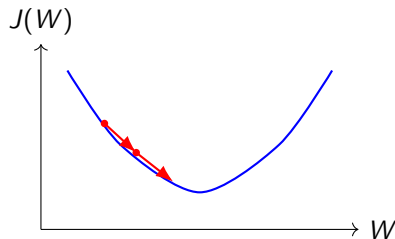
$$e_k(p) = \gamma_{d,k}(p) - \gamma_k(p)$$

Weights are updated in the direction that reduces the loss.

$$W \leftarrow W - \eta \frac{\partial J}{\partial W}$$

$$b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

- η : learning rate
- J : loss function
- $\frac{\partial J}{\partial W}$: gradient of loss with respect to weights



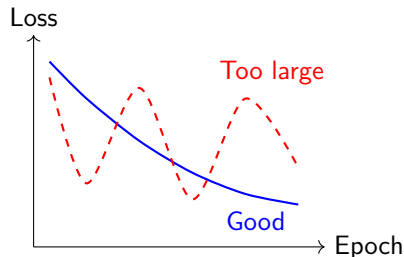
Learning rate controls the step size during optimization.

Too small

- Slow training
- Many epochs required

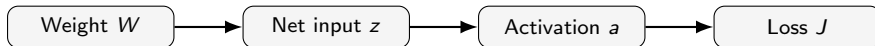
Too large

- Training may oscillate
- Loss may diverge



Backpropagation applies the chain rule layer by layer.

$$\frac{\partial J}{\partial W^{[l]}} = \frac{\partial J}{\partial a^{[l]}} \frac{\partial a^{[l]}}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial W^{[l]}}$$



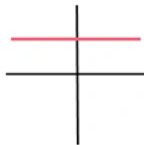
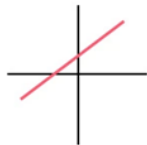
The gradient tells how much each weight contributed to the final error.

- 1 Initialize weights and biases.
- 2 Perform forward propagation.
- 3 Compute loss.
- 4 Compute output layer gradients.
- 5 Propagate gradients backward through hidden layers.
- 6 Update weights using gradient descent.
- 7 Repeat for many epochs.

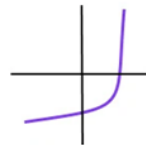
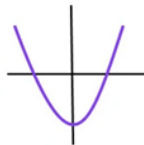
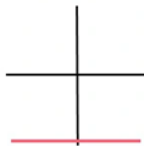
Epoch

One complete pass through the training dataset.

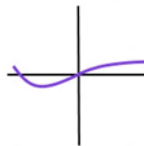
Different Types of Activation Functions

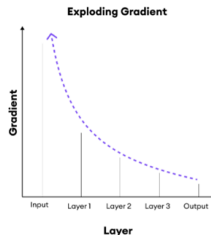
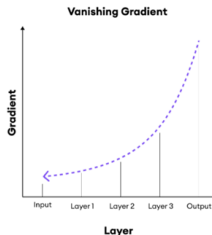


Linear Functions



Nonlinear Functions





Vanishing gradient

- Gradients become very small.
- Early layers learn slowly.
- Common with sigmoid/tanh in deep networks.

Solutions: Include ReLU, good initialization, batch normalization, gradient clipping and residual connections.

Exploding gradient

- Gradients become extremely large.
- Training becomes unstable.
- Common in deep or recurrent networks.

Reference: <https://www.superannotate.com/blog/activation-functions-in-neural-networks>

C 1: The Dream

- Scientists wanted machines to learn like the human brain.

C 2: The Gradient Problem

- Each weight contributes differently to the final prediction error.

C 3: The Chain Rule

- The chain rule combines these derivatives through the network.

C 4: Need for Automation

- Researchers needed automatic gradient computation.

C 5: TensorFlow

- Its early strength was static computation graphs and production use.

C 6: PyTorch

- Dynamic graphs made coding and debugging more intuitive.

C 7: Competition and Convergence

- TensorFlow adopted eager execution and a more Pythonic interface.
- PyTorch added compilation, serving and deployment tools.

C 8: The Common Foundation

- Both use reverse-mode automatic differentiation.
- Both apply the chain rule on a computation graph.

Final Message

ANNs provided the learning model, backpropagation provided the mathematics, automatic differentiation provided the algorithm, and TensorFlow and PyTorch made the complete process practical and scalable.